

Autonomous and Mobile Robotics Exam

Group 30 Report

Gaggioli Alessandro
Gamberini Mattia

Task 1

Objective

Task 1 requires deploying the TIAGo robot within an initially completely unknown simulation environment and performing autonomous mapping using Simultaneous Localization and Mapping (SLAM) techniques. The first task is to generate a reliable occupancy grid map that will serve for all subsequent navigation tasks. The execution flow consists of the following sequential states:

1. Start the Gazebo simulation in the assigned environment (group30 world)
2. Command the robot's manipulator arm to a defined home joint configuration to ensure it minimizes the robot's physical footprint to avoid collisions during navigation.
3. Activate the Nav2 stack in SLAM mode, enabling the system to simultaneously build the map and track the robot's pose.
4. Start autonomous exploration of the environment to cover and map the area.

The only responsibility of the custom node is to secure the arm in the home position, after which exploration is delegated to *explore_lite* and mapping to *slam_toolbox*.

Architecture

Component	File	Function
Launch	task1.launch.py	Launch simulation, Nav2/SLAM, arm tuck, start exploration
Task1Manager	task1_manager.py	Entrypoint node: instantiates ArmController + Task1StateMachine, starts run_task
Task1StateMachine	task1_state_machine.py	Non-blocking state machine that moves the arm to home and waits for the motion to complete
ArmController	tiago_arm.py	Interface to MoveIt2 and head via JointTrajectory
slam_toolbox	Nav2 bringup	Incremental map construction + TF map → odom
explore_node	explore_lite	Selecting boundaries and sending navigation goals
Nav2 params	tiago_nav2.yaml	Complete Nav2 configuration (controller, planner, costmap, behavior, SLAM) shared with Tasks 2/3
Common Launch file	launch_common.py	Shared includes tiago_sim() and nav_bringup(slam=True)

task_runner.py

Shared with task 2 and 3.

1. Creates a `threading.Event` (`executor_ready`) to coordinate threads.
2. Starts `state_machine.run()` on a dedicated daemon thread.
3. Runs `executor.spin_once()` on the main thread until `state_machine.finished` is not true, then shuts down the node.

The logic runs on a thread separate from the one that processes subscriptions, service clients, and action futures, so no operations block the loop. The state machine waits for `executor_ready` and then it sleeps for 10 seconds to give the other components time to complete the startup.

task1.launch.py

The sequence is controlled via a `RegisterEventHandler` on `OnProcessExit`

- `task1_manager` is started with the simulator (`tiago_sim()`) and `slam_nav` (`nav_bringup(slam=True)`).
- When `task1_manager` terminates (arm in home position) a `TimerAction` with a period of 5 seconds waits for a brief settling period and then starts exploring node. This ensures that `explore_lite` starts only after the arm has already been tucked, preventing exploration with the arm in motion.

Task1StateMachine

The state machine uses the shared methods (`state_0_arm_tuck`, `state_1_wait_arm`) from `CommonStates` used in Tasks 2 and 3 supervising the arm tuck without blocking the rest. The state machine transitions are driven by flags updated every tick by `arm.update_flags()`.

Tuck arm to home

At startup, when the executor is stable, the controller commands the arm to the home configuration on `MoveIt2`. The send time is recorded, and the system immediately transitions to State 1. The arm in the spawn pose is extended, it would collide with the obstacles, compromising the navigation and mapping.

Wait for arm motion

state_1_wait_arm: at every tick, the state of `MoveIt2` is read via `arm.update_flags()`, which manages two one shot latches:

- *motion_started* when `MoveIt2` enters `EXECUTING`
- *motion_done* when `MoveIt2` returns to `IDLE` after execution

Two independent timeout guards are applied:

- `PLANNING_TIMEOUT`: the goal was sent, but the motion never started → planning failure or `MoveIt` not ready → return to State 0 and retry.
- `EXECUTION_TIMEOUT`: the motion started but stalled halfway → return to State 0.

When `motion_done` is true, the machine goes to the terminal state.

Terminal State

Once the terminal state is reached, the machine sets `finished = True`, the `run_task()` loop exits the spin and destroys the `Task1Manager` node.

Nav2 stack in SLAM mode

The `tiago_nav2.yaml` file defines the entire Nav2 stack and is shared with Task2 and 3. In SLAM mode, the active localizer is `slam_toolbox`. `slam_toolbox` builds the map and publishes `map` $\rightarrow$ `odom`.

`slam_toolbox`

- **mode:** mapping starts the system in map-building mode.
- **scan_topic:** `/scan_raw`, **odom_frame:** `odom`, **map_frame:** `map`, **base_frame:** `base_footprint` define the inputs and frames for the mapping.

planner server

Generates the global path, important for planning in areas not yet observed during exploration.

- **planner_plugins:** `["GridBased"]` uses `NavfnPlanner` (classic grid-based planner).
- **use_astar:** `true` selects A* for more informed plans compared to Dijkstra.
- **tolerance:** `2.00` provides flexibility when the exact position of the goal is difficult to reach.
- **allow_unknown:** `true` is necessary because the robot must be able to plan even toward unknown regions, where the boundaries lie.

Global costmap

- **global_frame:** `map`, **robot_base_frame:** `base_footprint`, **robot_radius:** `0.28`.
- **update_frequency:** `7.0`, **publish_frequency:** `5.0`
- **track_unknown_space:** `true`
- **plugins:** `["static_layer", "obstacle_layer", "inflation_layer"]`
- **inflation_layer:** **inflation_radius:** `0.7`, **cost_scaling_factor:** `3.5`

Local costmap

- **global_frame:** `odom`, **robot_base_frame:** `base_footprint`, **width/height:** `5`, **resolution:** `0.05`.
- **update_frequency:** `10.0`, **publish_frequency:** `10.0`
- **plugins:** `["voxel_layer", "inflation_layer"]`, with **z_voxels:** `40`, **min_obstacle_height:** `0.05`, **max_obstacle_height:** `2.0`

- **scan** topic: `/scan_raw`, **pointcloud** topic: `/head_front_camera/depth_registered/points`
- **inflation_radius**: 0.45, **cost_scaling_factor**: 10.0:

The global map is used by the global planner to find the entire path from the starting point to the destination. The local costmap is used by the local controller (DWB) to follow the global path avoiding obstacles. The local costmap is a 5x5-meter “rolling window” (`rolling_window`: true) centered on the robot, which moves along with it.

Global costmap is set to map, anchored to the static map of the environment. Local Costmap is set to odom, it uses odometry because it requires short term data.

`track_unknown_space`: true allows the costmap to handle the unknown spaces and use them as free space (to explore it).

`inflation_radius`: max distance within the extra cost is applied around an obstacle/wall. Global (0.7) such as the global planner will seek central routes, far away from walls and corners. Local (0.45) allowing the local controller to navigate through narrow passages.

`cost_scaling_factor` controls the rate at which the cost decreases within the inflation radius. The higher the value, the faster the cost decreases as it moves away from the obstacle. Global (3.5) decreases slowly, paths near obstacles are heavily weighted, forcing the robot to take large paths. Local (10.0) drops faster, allowing the robot to perform maneuvers slightly closer to obstacles, since the cost gradient drops just a few centimeters from the actual obstacle.

Autonomous Exploration

The NavGoal decision is handled by `explore_lite`, which analyzes the boundaries on the SLAM map and sends the goals to Nav2.

- **planner_frequency**: 0.33 (Hz) frequency at which the node searches for new frontiers.
- **progress_timeout**: 60.0 is the maximum time (in seconds) allowed for the robot to demonstrate progress toward a boundary. If the robot gets stuck or fails to approach the target for 60 seconds, the boundary is added to a “blacklist” and the robot moves on to the next one.
- **gain_scale**: 2.0, multiplier for the size (information gain) of the boundary. When set to twice the value of **potential_scale**: 1.0, the robot will prioritize larger boundaries, accepting to reveal larger portions of the map. **orientation_scale**: 0.0 ignores orientation in selection.
- **min_frontier_size**: 0.25 is the minimum size (in meters) required for a boundary to be considered an explorable frontier.
- **return_to_init**: false: once exploration is complete, the robot does not return to the starting point.

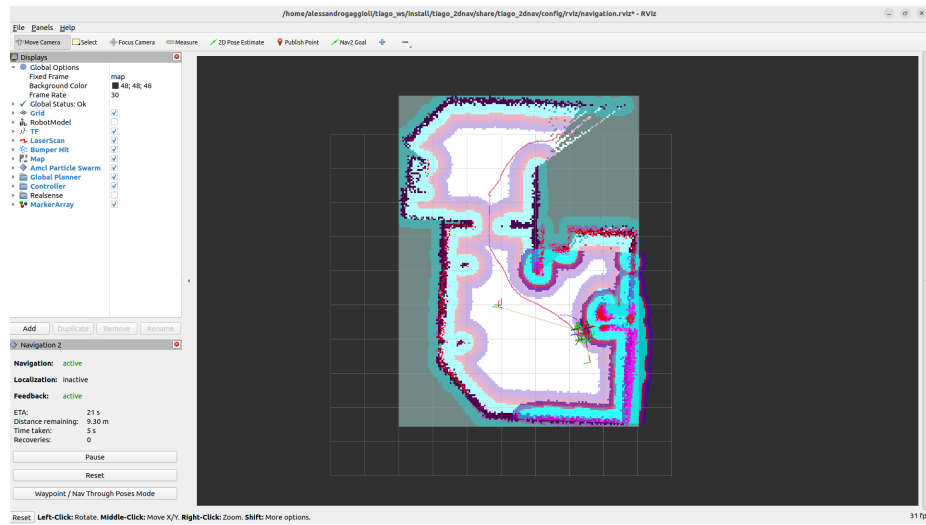


Figure 1: exploration in progress

Mapping is considered complete when `explore_lite` can no longer find any explorable frontiers. At that point, the map can be saved using the standard Nav2 tools (`map_saver`).

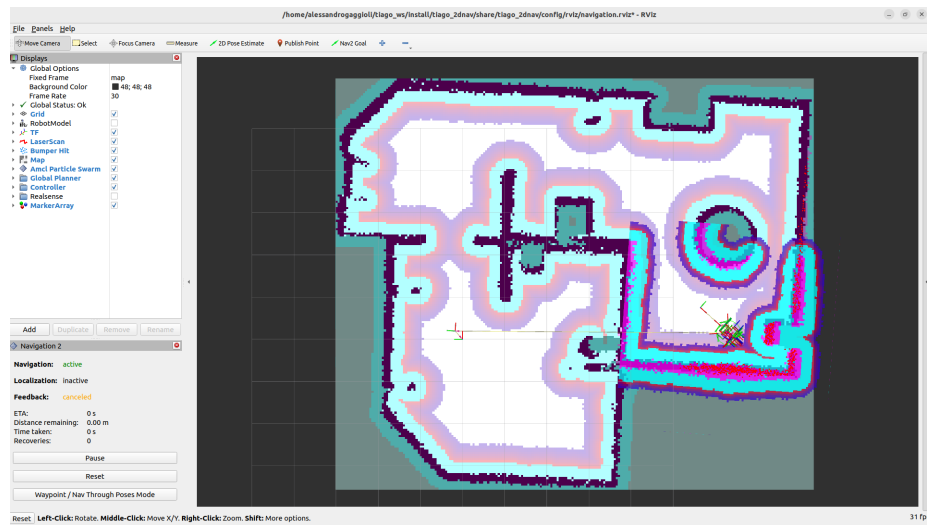


Figure 2: full map - All frontiers traversed/tried out, stopping

Task 2

Objective

Task 2 starts with the TIAGo robot spawned in a random and unknown position within a known map, built in Task 1. The robot does not know its initial pose and must complete autonomously:

1. Bring the arm in home position and tilt the head.
2. Locate itself on the saved map using AMCL (global localization).
3. Explore the environment with random waypoints until it detects both ArUco markers (the pick marker with ID 26 and the place marker with ID 238).
4. Navigate to pick marker approach pose.
5. Navigate to place marker approach pose.

The entire workflow is implemented as a non blocking state machine. The task is performed by composing five independent subsystems (arm/head, navigation, AMCL, localization, ArUco detection, waypoint sampling) coordinated by the state machine.

Each action is launched asynchronously and at each iteration of the loop the state machine reads the flags updated by `update_flags()` methods to decide if advance, retry or handle a timeout.

Localization is organized into 4 substates (particle scatter, costmap cleaning, spin, waiting for convergence) and Search Phase (navigation) is divided into 3 substates (sample nav goal, nav, pan) that repeats until both markers have been seen.

Architecture

The main Task2Manager node (`task2_manager.py`) instantiates five components, each in its own file, plus the state machine:

Component	File	Function
ArmController	<code>tiago_arm.py</code>	Arm via MoveIt2 + head tilt/pan via JointTrajectory
NavClient	<code>task2_nav.py</code>	Nav2 Client NavigateToPose + <code>/nav_vel</code>
AmclLocalizer	<code>task2_amcl.py</code>	AMCL global localization + Nav2 spin + costmap cleanup
ArucoTracker	<code>task2_aruco.py</code>	marker detection + calculation/broadcast of approach poses
CostmapSampler	<code>task2_costmap.py</code>	Sampling waypoints from global costmap
State Machine	<code>task2_state_machine.py</code>	state machine

Shared states are implemented in `common_states.py`. Low level helpers are implemented in `state_helpers.py` (class `StateRunners`). KDL and quaternion functions are in `task2_kdl_helpers.py`. All constants are centralized in `constants.py`

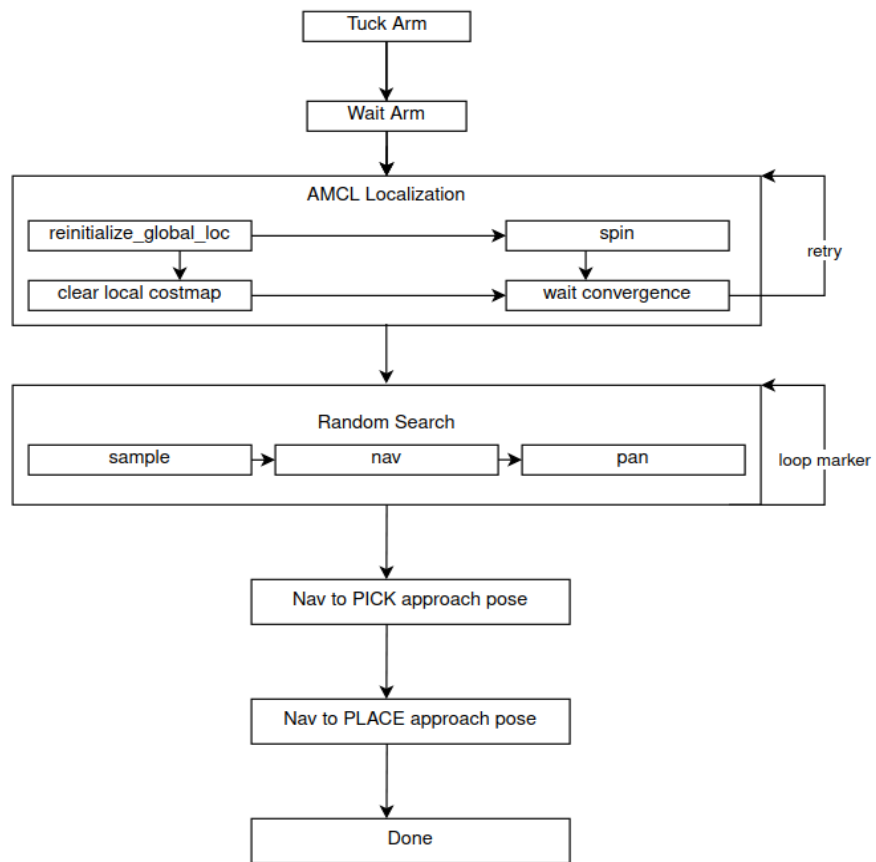


Figure 3: Task2 flowchart

AMCL global localization (state_2_amcl_localization)

- **request_global_localization()** calls the `/reinitialize_global_localization` service via an asynchronous client service. AMCL redistributes its particles across the map, abandoning any previous estimates. After the call, wait 2 seconds for AMCL to redistribute the particles, then move to the next substate.
- **Clear local costmap** Clear the local costmap with `/local_costmap/clear_entirely_local_costmap` and wait 0.5 s.
- **spin** 2π increases `localization_spin_count` and sends a goal to the Nav2 `/spin` action with target yaw = 2π and `time_allowance` = 30 s. The full rotation exposes the lidar to all directions, providing AMCL with enough scans to converge.
- **Wait for convergence** Each tick `update_amcl_flags()` reads the covariance from `/amcl_pose` at `x,y,yaw`. Convergence is declared when: $\sigma_{xx} < 0.10$ and $\sigma_{yy} < 0.10$ and $\sigma_{yaw} < 0.07$. The particle filter always maintains a tail of randomly scattered particles, the thresholds declare convergence as soon as the main mass clusters.

If it converges (even before the end of spin): the active spin is cancelled and the state transits to State 3, `update_amcl_flags()` calls `clear_global_costmap()`.

While AMCL reported an incorrect pose, the obstacle layer projected lidar readings in the wrong locations, creating shadow obstacles that would remain until the robot passed over them. Clearing the global costmap allows to pass into areas that are actually clear.

If the spin ends without convergence: if the robot performed 3 spins, it transits to State 3 anyway (hopefully AMCL will be better along the navigation), otherwise returns to spin 2π .

task2_amcl.py

- `request_global_localization()`, `clear_local_costmap()`, `clear_global_costmap()` service clients for `std_srvs/Empty` and `ClearEntireCostmap`.
- `send_spin()` / `cancel_spin()` Nav2 action client for `/spin`.
- `update_spin_flags()` handles the spin future pipeline, setting `spin_done` and `spin_succeeded` (status 4 = SUCCEEDED).
- `update_amcl_flags()` evaluates convergence from the covariance and, upon first convergence, clears the global costmap.

Random search (state_3_random_search)

The robot explores by navigating to random waypoints. Each waypoint it reaches, it pans its head to look around. The state exits when both approach poses (pick and place) have been latched by ArUco tracker.

- **sample** if the costmap has not yet arrived, it waits. Otherwise, it samples a waypoint with `sampler.sample_random_xy()`. If it returns None (all cells visited) the visited_waypoints list is emptied and the state transits to the next substate. Otherwise, the selected waypoint is added to visited_waypoints, and the Nav2 goal is sent.
- **nav** waits for the goal. If successful, the pan starts. Upon failure, the search returns to sample. A timeout guard is active, when it expires, the goal is deleted and the search returns to sample.
- **pan** The head sweeps through the search pan positions (-0.6, 0.6, 0.0) [rad] staying for 1.5 s at each. If the sweep completes without seeing any markers, return to sample.

We tried before the spin instead of the head pan. The per-waypoint spin was fragile, DWB frequently aborted the rotations, and each abort disturbed the AMCL. The head pan does not move the base, it is not affecting the AMCL estimate.

task2_costmap.py `sample_random_xy()` does the following:

1. Free cell mask: only cells with cost exactly 0 (navigable, not inflated)
2. Margin from edges: reset cells with edge margin (=1.0) from the edge of the map. The NavFn global planner expands a potential field around the goal; if it reaches pixels outside the map, Nav2 logs worldToMap failed, 1.0 m is well beyond the inflation range, so the planner never touches out of bounds pixels
3. Robot pose: via TF map $\leftarrow$ base_link
4. Distances: euclidean distance of each free cell to the robot.
5. Rejection of visited: cells within visited_radius of any waypoint already attempted are discarded, to avoid repeatedly returning to the same points.
6. Distance cap: candidates are the cells not visited and within soft_max_range (to prefer nearby waypoints). If none are available, all non visited cells are accepted.
7. Exponential weighting: $w = \exp\left(-\frac{d}{preferred_range}\right)$. Nearby waypoints are preferred, but sampling remains probabilistic.
8. Extraction: `np.random.choice` on the normalized distribution.

ArUco detection (task2_aruco.py) Two instances of `aruco_single` (package `aruco_ros`), one for marker ID 26 (PICK) and one for 238 (PLACE), both 0.25 m, with `reference_frame=""`. With this setting, the node publishes the marker pose in the camera's optical frame without applying TF: the compositing in the map frame is done entirely in our code.

Handling a detection:

1. Convergence: ignore any detection until AMCL has converged.
2. Transform: conversion of the marker in the camera into KDL Frame.
3. Freeze: if the pose is already stored and refresh is disabled (`freeze()`), it is ignored.
4. Distance filter: detections beyond a fixed range discarded (to prevent perspective projection errors)
5. Keep closest: if the marker has already been seen, updates it only if the new distance is smaller than the previous one.
6. Time: look for the TF map $\leftarrow$ camera at the exact timestamp of the message.
7. Transform: the marker in map pose is obtained with `frame_map_cam * aruco_in_cam`

During searching, the approach pose is continuously updated (keep-closest). Upon transition to State 4, `aruco.freeze()` sets `_refresh_approach_poses = False`: from that moment on, the Nav2 goal no longer follows a moving estimate, avoiding continuous replanning during the approach.

Approach pose computation:

1. Extract the z direction of the marker in the map
2. Projected onto the horizontal XY plane and normalized.
3. set the approach position at `approach.distance` from the marker along the direction.
4. The yaw is $\arctan\left(\frac{\text{marker.y} - \text{approach.y}}{\text{marker.x} - \text{approach.x}}\right)$, so the robot points towards the marker.
5. Orientation is set to `Rotation.RPY(0,0,yaw)`

The pose is broadcast as TF (`aruco_pick_approach / aruco_place_approach`) as PoseStamped for the Nav2 goal.

Navigate to pick approach (state_4_pick)

Send/Wait/Retry mechanism.

- **send:** If there is no movement command in progress, the system takes the target coordinates and sends the Nav2 goal.
- **wait:** wait for the goal, if the goal is succeeded, log it and move to the next state (5).
- **retry:** If fail, log the error and return to send.

Navigate to place approach (state_5_place)

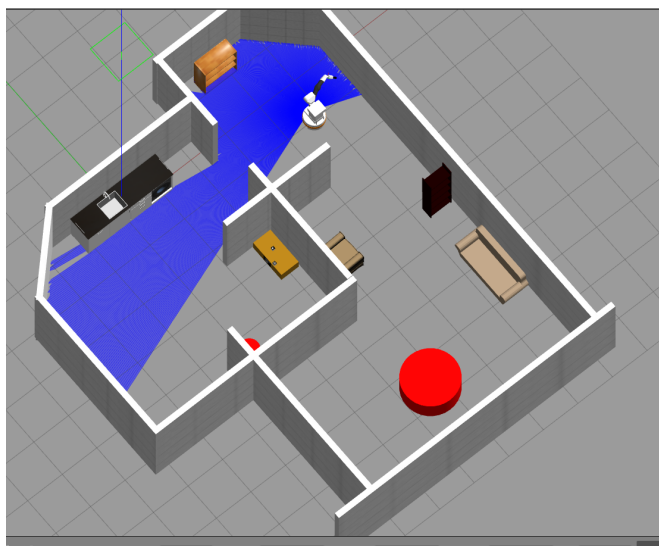
Identical to State 4, but towards place_approach_pose, with next_state = 6. Same timeout and same retry logic.

Done (state_6_done)

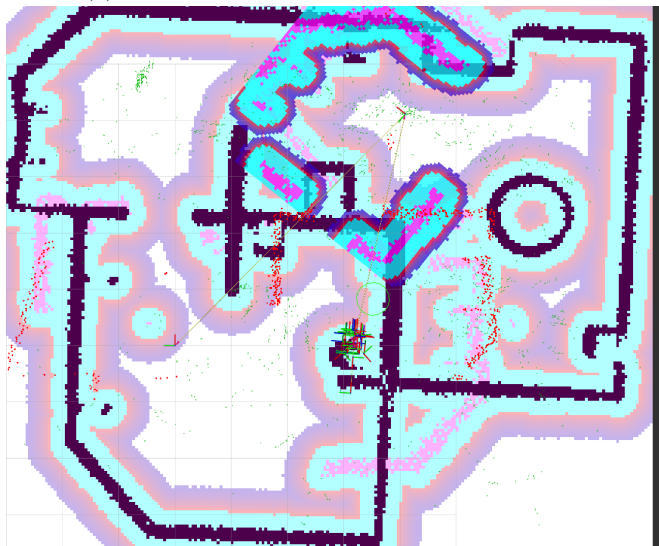
Set self.finished = True. The run_task() loop exits the spin and shuts down the node



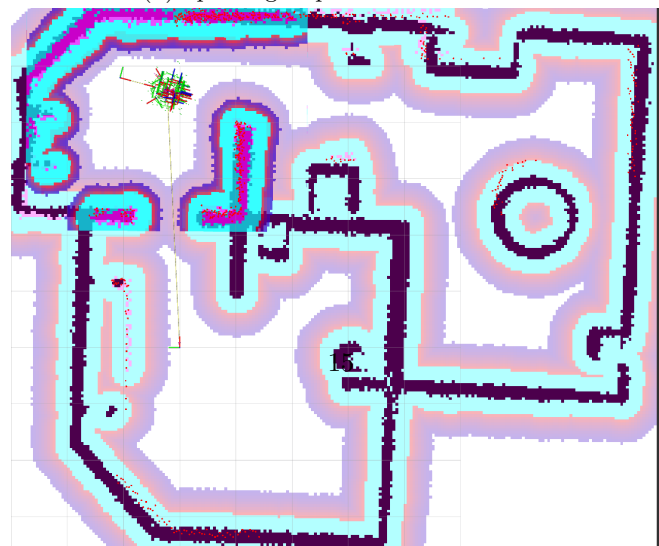
Figure 4: Robot moving towards pick approach (after detected markers))



(a) The robot is spawned in a random position



(b) Spinning to perform localization



(c) Robot is localized in the map

Task 3

Objective

Task3 extends Task2 with a pick and place cycle of two cubes marked with ArUco (ID 63 and ID 582). Starting from the robot in a random and unknown pose, the system must run the entire flow of Task2, detect the marker on the cube to be picked, grab the cube, navigate to the approach pose, and deposit the cube. Repeat for the second cube. As in Task2, everything is implemented with a non blocking state machine.

Task3 uses the components of Task2 and adds three new ones (gripper, link attacher, cube marker tracker).

The state machine uses an IntEnum Phase whose values coincide with the states of CommonStates, to use the states of Task2 and add manipulation sub phases. States 0-3 are identical to Task2. State4 does not terminate, but starts the pick sub flow. State 5 starts the drop sub flow.

Architecture

The Task3Manager node (`task3_manager.py`) instantiates the components of Task2 plus the three new.

Component	File	Function
Task 2 Components	task2_.py, tiago_arm.py	Nav2, AMCL, ArUco, MoveIt2 arm, costmap sampler
GripperController	tiago_gripper.py	Opening/closing the gripper
LinkAttacher	task3_link_attacher.py	Asynchronous services /attachlink and /detachlink (IFRA plugin)
CubeTracker	task3_cube_tracker.py	Markers on cubes
Task3StateMachine	task3_state_machine.py	Complete state machine

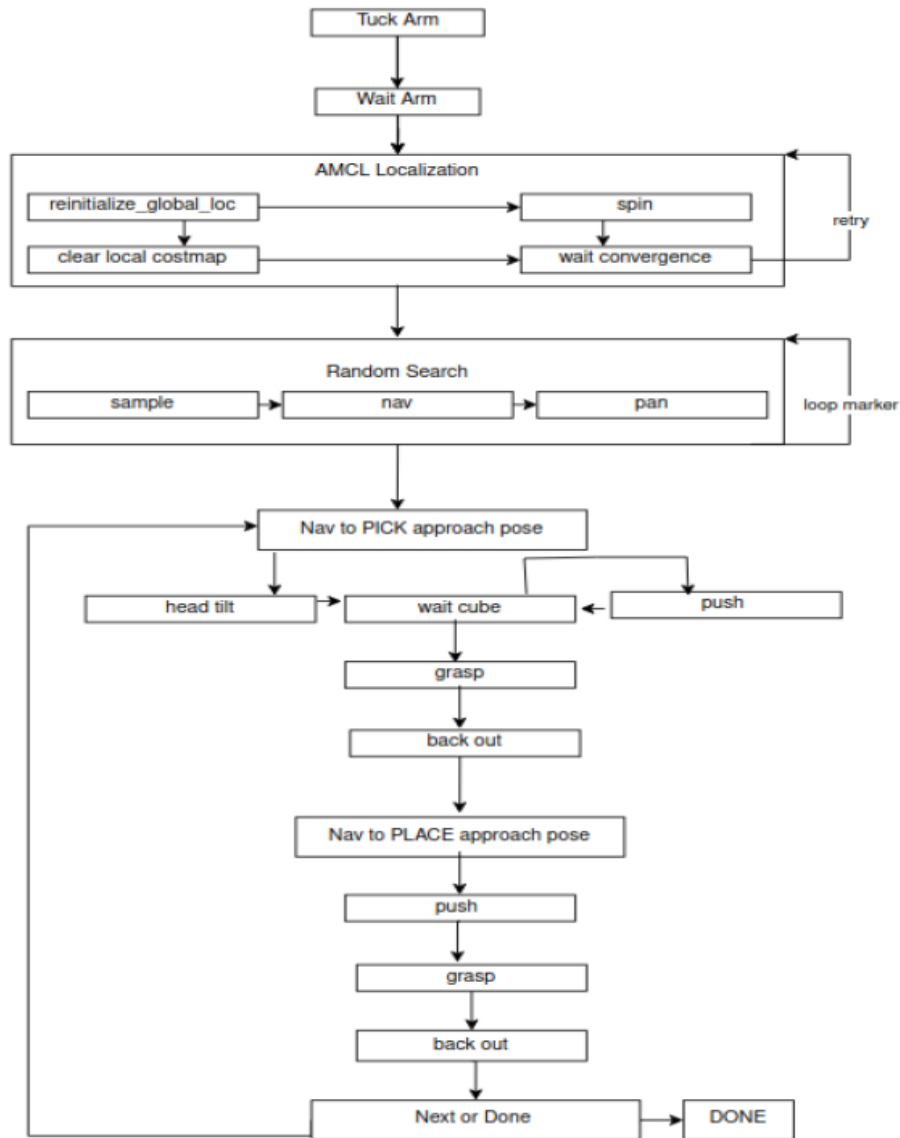


Figure 6: Task3 flowchart

Grasp and Drop Phases

Head tilt (`_phase.head_tilt`)

- Enable CubeTracker: while searching for Task2, it was disabled to avoid storing noisy perspective projection poses when viewed from far away.
- Tilt head, the marker approach pose has already aligned the robot to the

surface, so the cube should be facing forward.

- Initialize head scan state and switch to wait_cube

Wait Cube (`_phase_wait_cube`)

Three behaviors:

- Marker not yet seen: A head scan starts, to bring the cube into the field of view of the robot.
- Marker seen, but cube far away: the pose of approaching the table typically leaves the cube outside the arm workspace. Send a cube Nav2 goal (`_send_cube_approach_nav`) and switch to wait_cube_nav.
- Marker seen and robot approached: Freeze the tracker (`freeze()`), compute the grasp poses (`_precompute_grasp_poses`) and start the grasp sequence

Freezing before grasp is required because late noisy detection could move the grasp laterally.

Wait Cube Nav (`_phase_wait_cube_nav`)

Waits for the outcome of the cube goal issued in wait_cube:

- Success: Nav2 stopped at inflation limits. It tilts its head back down, sets its `_push_mode = pick` and switch to push.
- Fail: Set `_cube_approached = True` and return to wait_cube to attempt the catch anyway

`_send_cube_approach_nav`: The goal sets the `base_link` to a `safe_nav_distance` from the cube center, along the normal to the pick marker, not along the cube direction. Using the cube direction could position the robot too close to the surface, where the footprint comes within the inflation radius, and DWB rejects any trajectory.

Push (`_phase_push`)

At each tick, it reads the TF of the `base_link` w.r.t. the map, calculates the distance from the reference marker and publishes a Twist to `/nav_vel`. Advances at 0.15 m/s until it is within a `stop_dist`. The angular velocity is proportional to the heading error, allowing the robot to approach the target frontally. It uses a `/nav_vel` and not a `/drive_on_heading` or `/cmd_vel` because the drive on heading was fragile and aborted by behaviors, `cmd_vel` is absorbed by `velocity_smoother` when Nav2 is idle.

Run Sequence - Grasp and Drop

The driver executes the current step. When it returns True, it advances, resets the `_step_sent` and when the list is finished, it executes the `on_complete` callback.

Grasp sequence:

1. Open the gripper
2. Arm to pre grasp: above the cube center, with the vertical descent orientation
3. Grasp arm: `cartesian = True` so the end effector descends in a straight line (a joint space plane would curve the trajectory).
4. Close the gripper
5. Attachlink: to create a link between the gripper and the cube
6. Lift: reverse of descent, `cartesian = True`.
7. Home positioning.

`on_complete = _grasp_complete`: Bring the head back to level and start `back_out("pick")`.

Drop sequence:

1. Pre drop arm: `_precompute_drop_poses` calculates the poses, pre drop is raised above the target
2. Drop arm: cartesian descent
3. Open the gripper
4. Detach link: erase the link between the gripper and the cube
5. Pre drop lift: to not collide with the cube
6. Home positioning.

Back Out (`_phase_back_out`)

The reverse of push, until `base_link` is not at least `SAFE_NAV_DISTANCE` from the reference marker, then moves to the next state (`NAV_PLACE` for pick, `NEXT_OR_DONE` for place)). After the push phase, the robot is within the inflation radius of the surface, preventing Nav2 from planning a path. So moving away in closed loop makes the robot able to move autonomously.

Grasp poses (`_precompute_grasp_poses`)

The pre-grasp is computed before the grasp descent. The pose is expressed in the `base_link` frame to avoid localization drift during manipulation. It first attempts to use the most recent observation in the camera frame (`cube_marker_in_camera`). If the cube is out of the field of view, it falls back to the marker pose stored in the map.

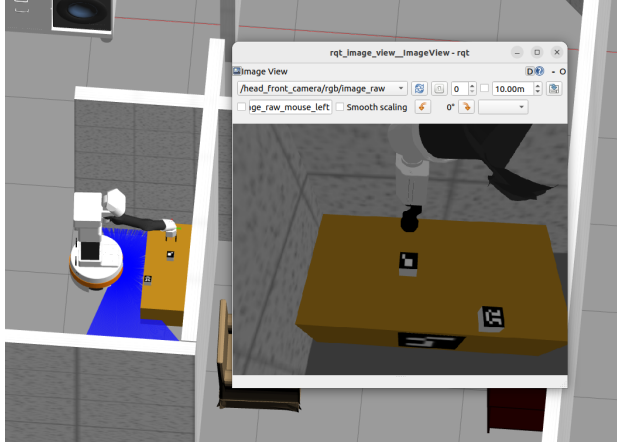
The marker is on the upper face of the cube, the center is `CUBE_TOP_TO_CENTER` = 0.035 m lower. GRASP is located at `cube_top.z + GRASP_Z_ABOVE_TOP`, PRE-GRASP is at `cube_center.z + PRE_GRASP_LIFT`.

The roll and pitch estimates from the ArUco algorithm are noisy. Using the full marker rotation would tilt the descent axis, causing the end effector to descend along an inclined path rather than vertically. So, it saves only the yaw component of the marker, and the final orientation is computed as:

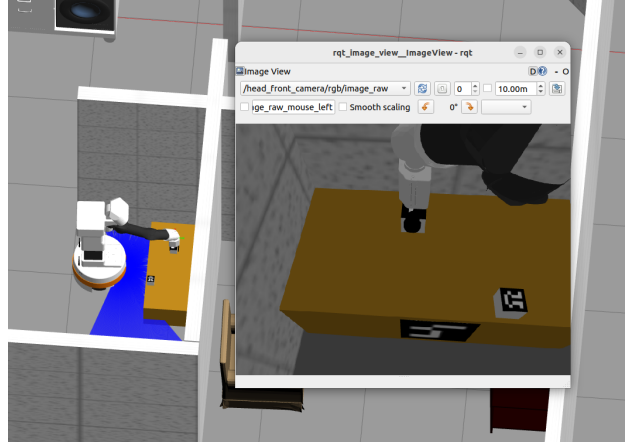
$$R_{grasp} = I * R_z(cube_yaw) * R_y(\frac{\pi}{2})$$

Drop poses (`_precompute_drop_poses`)

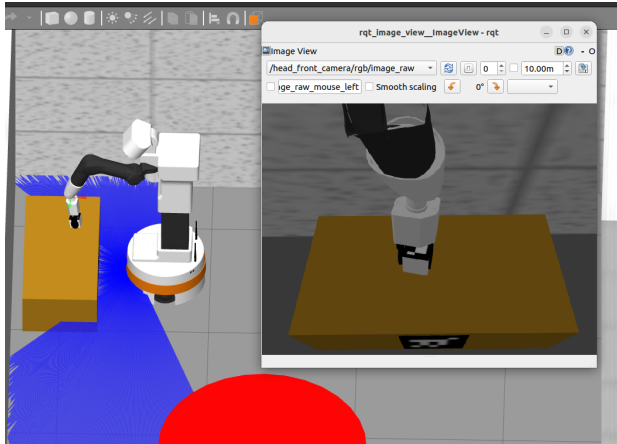
Compute the drop poses. The target point to release the cube is set in front of the robot (`place_forward_offset`), on the place surface. To ensure not releasing the cubes one over the other, possibly causing a collision, the targets are offset laterally (first cube on the left, second cube on the right). The z coordinate is set above the place surface to avoid collisions with it during the descent. For orientation, the pattern is the same as the one used for the grasp, it is set the pure rotation $R_y(\frac{\pi}{2})$, guaranteeing that the arm descends vertically before opening the gripper.



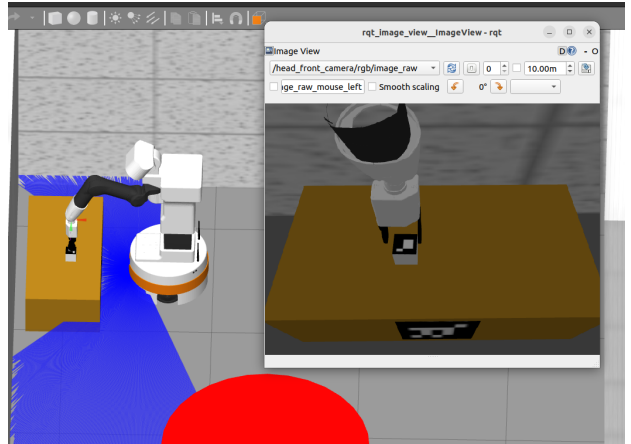
(a) Pre grasp



(b) Grasp



(c) Pre drop



(d) Drop